



COCKROACH BROWSER / VERSION 0.5.0-RC.1

A governed multi-engine runtime for AI agents

Full browsers for fidelity. A verified lightweight lane for compatible non-visual work. Explicit authority for both.

Author: Ajnas N B

Paper version: 1.2

Date: September 3, 2026

Concept DOI: 10.5281/zenodo.21701791

Software: AGPL-3.0-or-later

Paper: Creative Commons Attribution 4.0 International

Status: Implementation-backed technical white paper. The paper has not undergone independent peer review or independent security certification.

Contents

Implementation, authority boundaries, integrations, deployment, and verification.

Abstract	4
1. Problem and category	4
2. Contributions	5
3. Explicit nonclaims	5
4. Actors, assets, and trust boundaries	6
5. Architectural overview	6
6. Engine capability matrix	7
6.1 Chromium	7
6.2 Firefox	7
6.3 WebKit	8
6.4 Obscura	8
6.5 Lightpanda	8
6.6 Operator-supplied browsers and devices	8
7. Authorized sessions	9
8. Action contract and semantic observations	9
9. Bounded structured extraction	10
10. The AI agent boundary	10
11. Model gateway	10
12. Qarinah context and provenance	11
13. Maqam, crawler, and other integrations	11
14. Evidence and receipts	11
15. Team tenancy and admission	12
16. Resource governance	12
17. Network boundary	13
18. Deployment architecture	13
19. Distribution and release integrity	14
20. Verification status	14
21. Lightweight benchmark question	15
22. Benchmark identity	15

23. Benchmark method	15
24. Benchmark results	16
25. Interpreting the result	17
26. Competitive landscape	17
27. Intended users and jobs	18
28. Adoption and go-to-market	18
29. Limitations and open risks	19
29.1 Windows SideBySide limitation	20
30. Reproducibility	20
31. Evidence roadmap	21
32. Conclusion	21
Appendix A. Bounded action kinds	22
Appendix B. Claim controls	23
Appendix C. Release and deployment checklist	24
Appendix D. Source and evidence map	24
Acknowledgements	24
References	25

Abstract

AI agents need browser capability because important work happens inside dynamic, stateful web applications. The same capability also creates an unusually broad authority boundary: a browser can read authenticated data, mutate external systems, upload or download files, execute page code, contact network destinations, and retain credentials. A useful agent browser therefore needs more than a headless launch command. It needs an explicit answer to which engine is being used, which operations are supported, which origins and effects are authorized, how much resource use is allowed, what evidence is retained, and what happens when enforcement or teardown cannot be verified.

Cockroach Browser is an open-source, local-first, multi-engine browser runtime for AI agents. It routes full-fidelity work to Chromium, Firefox, or WebKit and can route compatible non-visual DOM and JavaScript work to an explicitly installed experimental lightweight engine. A machine-readable engine manifest reports supported, experimental, and unsupported capabilities before launch. The governed runtime binds each session to a purpose, origins, actions, effects, approvals, finite budgets, authenticated identity, evidence, and lifecycle controls. Raw Playwright, Puppeteer, CDP, WebDriver BiDi, WebDriver, and Appium surfaces remain available to trusted operators without being misrepresented as bounded agent actions.

The release-candidate architecture also includes bounded structured extraction, a finite-step model-directed agent, an OpenAI-compatible model gateway, cited Qarinah context that cannot grant browser authority, exact Maqam approval integration, team ownership and roles, admission limits, process-tree resource monitoring, content-addressed evidence, and hash-linked receipts. The source-derived capability registry contains 130 entries: 119 available surfaces, 11 host-backed adapters, and no planned entries. The bounded action contract contains 66 action kinds. These are implementation inventories, not performance or security scores.

This paper reports one canonical lightweight measurement without generalizing it. On a pinned Windows host, the reviewed Obscura 0.2.1 binary passed a 30 MiB target with a maximum complete owned-browser-process-tree RSS of 29,622,272 bytes, or 28.25 MiB, across 20 measured launches. A separately executed 25 MiB target failed with a maximum of 29,679,616 bytes. The Node coordinator was measured separately. Neither result is a whole-application, arbitrary-page, rendered-page, persistent-session, attached-session, or full-browser memory guarantee.

1. Problem and category

A browser is both an interpreter of untrusted content and a controller of consequential external systems. Traditional automation libraries expose powerful objects, but the host often has to build policy, identity, budgeting, evidence, teardown verification, and agent-safe semantics around them. A general desktop browser adds ambient profiles and machine state. A hosted browser can be useful, but it moves execution, credentials, and evidence into another operator's service boundary. A narrow HTTP scraper uses fewer resources, but cannot complete stateful or interactive workflows.

Cockroach Browser occupies a specific category: a governed, local-first, multi-engine browser runtime for AI agents. It is not a new rendering engine. It composes established full-browser engines, experimental independent lightweight engines, automation protocols, policy, resource enforcement, and evidence into one routed contract.

The central design principle is separation. Browser capability is separated from ambient machine authority. Governed actions are separated from unrestricted operator automation. Memory is separated from authorization. Browser evidence is separated from governance approval. Lightweight efficiency is separated from full-engine fidelity. Application telemetry is separated from kernel-enforced resource isolation.

2. Contributions

The 0.5.0-rc.1 development tree contributes the following implementation-backed elements.

- One explicit engine-selection contract spanning Chromium, Firefox, WebKit, Obscura, and Lightpanda manifests.
- Per-engine capability assessments with supported, experimental, and unsupported states.
- Fail-closed preflight for unsupported actions and explicit opt-in for experimental capabilities.
- No silent engine substitution inside an authorized session.
- A bounded runtime with explicit origins, action kinds, effects, exact approvals, finite budgets, and challenge pauses.
- A separate unrestricted operator layer that re-exports pinned Playwright and Puppeteer APIs and supplies raw CDP, BiDi, WebDriver, Appium, testing, code generation, and fleet adapters.
- Snapshot-scoped semantic references and fresh observations after agent actions.
- Host-side structured extraction from untrusted HTML with aggregate and category-specific limits.
- A finite-step agent loop with schema validation, tool-result bounds, deadlines, context compaction, evidence anchors, and cited history.
- A Qarinah record and retrieval boundary that preserves provenance while refusing to turn memory into authority.
- Team ownership, owner/operator/viewer roles, authenticated actor tokens, durable access updates, and concurrency-safe session admission.
- Continuous PID-observable owned-process-tree RSS, CPU-time, duration, and process-count monitoring with sticky terminal failures.
- Content-addressed evidence and hash-linked action receipts.
- A reproducible, artifact-hashed lightweight resource proof that publishes both a passing target and a failing target.

These contributions describe this implementation. They do not establish novelty in the legal, patent, or universal market sense.

3. Explicit nonclaims

This paper does not claim that Cockroach Browser is the only or first browser for agents, the fastest browser, the lightest browser, or a replacement for every browser-automation product.

It does not claim that every browser feature fits in 28 MB, that the whole application uses 30 MiB or less, or that Chromium, Firefox, WebKit, Lightpanda, the Node coordinator, arbitrary pages, rendered pages, attached sessions, or persistent profiles meet the measured Obscura result.

It does not claim that a renderer was disabled. The measured rendering policy is visual-actions-denied: Cockroach Browser rejected screenshot capture during capability preflight. That policy does not prove that the upstream Obscura binary omitted its renderer or received a renderer-disable switch.

It does not claim complete network isolation, a secure sandbox, containment of every browser protocol, universal site compatibility, CAPTCHA bypass, access-control bypass, stealth infrastructure, residential proxies, global regions, an operated hosted fleet, independent security certification, regulatory compliance, customer adoption, production reliability, or cost savings.

It does not claim comparative memory, speed, cost, accuracy, or throughput leadership. No same-host, same-workload, pinned cross-product comparison is reported here.

4. Actors, assets, and trust boundaries

The deployment owner installs the software and controls configuration, storage, process identity, network exposure, browser binaries, containers, and secret management.

The host application creates sessions and defines authoritative inputs. It decides which origins, actions, effects, profiles, credentials, providers, and resource ceilings are exposed to an agent.

The agent consumes bounded observations and proposes or requests actions. It does not own session policy, daemon identity, profile discovery, remote binding, or secret resolution.

The browser runtime evaluates policy, drives the selected engine, enforces application-level budgets, records evidence, and owns the lifecycle of managed processes.

The human reviewer handles login, consent, access challenges, and consequential approvals when the workflow requires human authority.

Maqam can evaluate policy, bind an approval to an exact canonical operation, dispatch through a registered driver, reject replay or changed input, and record a governance receipt.

Qarinah can record bounded metadata-only browser outcomes and retrieve cited project context. It cannot create sessions, change policy, approve actions, or dispatch browser operations.

Cockroach Crawler can map and extract bounded public web content before selected pages are handed to a browser for rendering, state, interaction, or evidence.

External engine and fleet providers remain distinct trust boundaries. Cockroach Browser reports their declared capabilities but does not turn an external service contract into a local security guarantee.

Page content, model output, downloaded files, retrieved memory, remote endpoints, and imported browser state are untrusted inputs. Host configuration, policy, approved secrets, and authenticated identity belong on the trusted side of the boundary.

5. Architectural overview

Cockroach Browser has two authority planes and two execution lanes.

The governed plane exposes 66 bounded action kinds through the embedded runtime, authenticated daemon, typed client, CLI, MCP proposal surface, model-directed agent, jobs, and authorized workers. Every operation remains subordinate to session policy and budgets.

The operator plane exposes exact upstream or protocol-level automation for trusted code. It includes pinned Playwright Core, Puppeteer Core, Playwright Test, code generation, CDP, WebDriver BiDi, WebDriver/Appium transport, and fleet adapters. This plane is intentionally powerful. It does not inherit bounded-runtime policy merely because it ships in the same package.

The full-fidelity lane uses Chromium, Firefox, or WebKit through Playwright. It is intended for rendering, visual evidence, complex interaction, browser state, downloads and uploads, frames, Shadow DOM, traces, HAR, video, and cross-engine validation.

The lightweight lane uses an explicitly installed and reviewed independent engine when the requested work matches its manifest. The current managed lightweight provider is experimental Obscura over a runtime-owned loopback CDP process. It targets compatible navigation, JavaScript, DOM inspection, bounded structured extraction, HTML element activation, and text-control value assignment. It is not treated as a transparent substitute for a full browser.

The main data flow is:

```

host policy and identity
  |
  v
engine manifest and action preflight
  |
  v
authorized session and finite budgets
  |
  v
browser action -> observation -> evidence -> receipt
  |
  +--> optional exact Maqam approval
  |
  +--> bounded Qarinah outcome and cited context

```

The architecture chooses an engine before executing work. If a lightweight session cannot admit a requested action, the caller receives an unmet capability result. Using a full engine requires a deliberate new or separately selected full-engine session; the runtime does not silently swap engines inside the governed session.

6. Engine capability matrix

The engine matrix is machine-readable and available without launching a browser. Each capability has a state, a note, and an engine identity. A successful preflight means the declared capability set is compatible with the requested actions. It does not create a session, grant an origin, supply credentials, or prove that a specific page will behave correctly.

6.1 Chromium

- Distribution and bounded integration: bundled Playwright engine, supported.
- Managed owned launch: supported.
- Playwright: supported.
- Puppeteer Core and CDP: supported on the operator surface.
- WebDriver BiDi: experimental through an operator-owned endpoint, not a managed bounded launch.
- Navigation, JavaScript, DOM, forms, same-origin frames, open Shadow DOM, CORS behavior, reviewed emulation, cookies, uploads, downloads, screenshots, video, traces, and runtime-owned persistent profiles: supported.
- PDF generation: supported in the bounded runtime.
- Reviewed unpacked extensions: supported in headed Chromium profiles.

6.2 Firefox

- Distribution and bounded integration: bundled Playwright engine, supported.
- Managed owned launch and Playwright: supported.
- Puppeteer and BiDi: experimental operator surfaces where upstream support applies.
- CDP: unsupported because this Firefox integration is not a CDP engine.
- Navigation, JavaScript, DOM, forms, same-origin frames, open Shadow DOM, reviewed emulation, cookies, uploads, downloads, screenshots, video, traces, and runtime-owned persistent profiles: supported.
- Bounded PDF generation: unsupported.
- The reviewed unpacked-extension launcher: unsupported because it is Chromium-specific.

6.3 WebKit

- Distribution and bounded integration: bundled Playwright engine, supported.
- Managed owned launch and Playwright: supported.
- Puppeteer, CDP, and managed BiDi: unsupported for this route.
- Navigation, JavaScript, DOM, forms, same-origin frames, open Shadow DOM, reviewed emulation, cookies, uploads, downloads, screenshots, video, traces, and runtime-owned persistent profiles: supported.
- Bounded PDF generation and the reviewed unpacked-extension launcher: unsupported.
- WebKit is useful for WebKit-engine behavior, but it is not a claim of exact Safari parity on every platform.

6.4 Obscura

- Distribution: external binary selected by the host.
- Maturity: experimental.
- Managed owned launch: experimental and allowed only for an explicitly installed, opted-in, conformance-tested binary.
- CDP, Playwright-over-CDP, Puppeteer-over-CDP, navigation, JavaScript, DOM, forms, frames, Shadow DOM, CORS behavior, emulation, interception, cookies, uploads, downloads, screenshots, and PDF: experimental and dependent on the selected build.
- Video, Playwright trace artifacts, persistent profiles, and reviewed unpacked extensions: unsupported by the current managed contract.
- Non-visual actionability means DOM activation and value assignment, not visual pointer fidelity.
- The canonical proof in this paper applies only to Obscura 0.2.1 with the recorded digest and fixture.

6.5 Lightpanda

- Distribution: external.
- Maturity: experimental manifest and preflight.
- Managed owned launch: unsupported in the current release candidate.
- CDP client compatibility is represented for planning and conformance work.
- The runtime fails closed because page-route interception does not establish a complete boundary for WebSockets, workers, WebRTC, WebTransport, and other egress.
- Enabling managed Lightpanda execution requires an engine-level or operating-system-level deny-by-default network boundary plus regression evidence.

6.6 Operator-supplied browsers and devices

- Safari on macOS can be reached through an operator-supplied W3C WebDriver endpoint.
- Android Chrome and WebView can be reached through W3C WebDriver or Appium.
- iOS Safari and WebView can be reached through W3C WebDriver or Appium.
- Remote browser farms can be represented through an authenticated provider that declares engines, regions, capacity, time-to-live, proxy classes, challenge modes, and live-view behavior.
- These are adapters. Cockroach Browser does not bundle a macOS Safari host, mobile simulator, emulator, device lab, global region network, proxy fleet, CAPTCHA solver, or live-view service.

7. Authorized sessions

Every governed session states a purpose, start URL, allowed origins, allowed action kinds, allowed effects, actions requiring approval, and finite budgets. The runtime requires at least one explicit allowed origin. Redirects and intercepted subresources are re-evaluated. Denied origins take precedence.

Private and loopback destinations are denied unless the deployment owner explicitly permits them for the workflow. Executable paths, CDP endpoints, launch flags, proxy credentials, extra headers, profile material, and remote listeners remain host-controlled inputs.

Named profiles are isolated. Storage state can be imported or exported only through explicit host operations. Runtime-owned persistent profiles require preparation and a single-writer lock. They cannot be combined with CDP attachment or imported storage state. The runtime does not discover ambient desktop profiles or cookies.

The default budget model covers actions, session duration, tabs, owned-process RSS, owned-process CPU time, downloads, uploads, snapshots, retained history, network entries, clipboard bytes, saved states, network rules, static fulfill bytes, intercepted bytes, and evidence bytes. A host should narrow every ceiling to the task rather than treating defaults as a security policy.

Login, consent, CAPTCHA, and access challenges pause the automated path. Challenge resolution is a critical execute effect requiring exact approval by default. An external resolver receives bounded challenge metadata, not raw browser control, cookies, storage, credentials, or the Playwright page. The runtime independently inspects the page after handoff and keeps the session paused when the challenge remains.

8. Action contract and semantic observations

The bounded action registry contains 66 action kinds. It covers navigation, tabs, pointer and keyboard input, forms, files, JavaScript under policy, structured inspection, emulation, cache and record maintenance, challenge handoff, paired capture, annotation, clipboard, network records and routes, storage state, cookies, page storage, screenshots, PDF, traces, extraction, and session tab locks.

Actions are classified by effect and risk before dispatch. Effects distinguish read, write, execute, upload, download, and credential-bearing operations. A session can admit observations while requiring a Maqam approval for an exact write, upload, download, credential use, arbitrary evaluation, or challenge resolution.

A semantic snapshot converts the current page into bounded text, structure, metadata, and stable-looking references scoped to the observed page revision. Those references are not ambient selectors. A page change invalidates stale references so an action cannot silently target a different element after navigation or mutation.

An ordered action batch does not bypass governance. Each attempted step receives its own validation, policy decision, execution status, and receipt.

The full-engine bounded runtime includes navigation, back, forward, reload, tabs and popups, click and double click, fill and type, keyboard and mouse operations, drag and drop, waits, same-origin frames, open Shadow DOM, dialogs, uploads, downloads, screenshots, Chromium PDF, traces, HAR, video, console and network records, deterministic emulation controls, cookies and storage, state checkpoints, audits, annotations, and visual comparison where the selected engine supports them.

Advanced unrestricted objects remain on the operator layer. Arbitrary handles, raw workers, every protocol command, extensions beyond the reviewed path, heap snapshots, CPU profiles, screencasting, and unrestricted routing are not relabeled as safe agent tools.

9. Bounded structured extraction

Structured extraction runs through a canonical host-side parser based on Parse5. Page-provided HTML is treated as untrusted input and is not asked to serialize the final evidence object. The extractor emits bounded combinations of visible text, sanitized HTML, Markdown, links, metadata, JSON-LD, and tables.

Traversal is iterative and has a hard depth guard of 512. The limits registry caps total extracted characters, category-specific characters, link count, metadata items, JSON-LD items and characters, table count, rows, columns, item size, and DOM nodes. The maximum aggregate content ceiling is 2,000,000 characters and the maximum DOM-node ceiling is 200,000. Lower defaults apply unless the host deliberately raises a field within the registry ceiling.

The runtime, MCP validator, schema, and tests use the same limits registry. Unknown limit fields fail validation. A category limit cannot exceed the aggregate session ceiling. URL output is normalized through safe URL handling rather than copied as trusted instructions.

Structured extraction reduces prompt volume for compatible work. It does not prove page truth, sanitize a downloaded document for arbitrary execution, or make an experimental engine equivalent to a full renderer.

10. The AI agent boundary

The built-in BrowserAgent is optional. A host can use its own planner and call the same governed runtime.

The built-in loop accepts one schema-valid tool action at a time. It rejects unknown action kinds, invalid payloads, multiple tool calls in one turn, and mixed action-and-finish output. Completion is exclusive: an assistant turn cannot both execute a browser action and declare the task finished.

After every non-snapshot action, the agent obtains a fresh bounded snapshot before another model decision. The loop retains action receipts, evidence identifiers, and citation anchors so that context reduction does not detach a conclusion from its observable basis.

Finite maxSteps prevents an unbounded planning loop. maxContextChars limits each serialized model turn. maxToolOutputChars bounds tool output. Compaction removes only complete older assistant/tool rounds and retains a digest plus receipt, evidence, and citation anchors. The agent does not expose hidden reasoning as evidence.

Deadlines cover the model call even when an upstream provider ignores an abort signal. A timed-out provider response cannot later be treated as an accepted action. Runtime policy remains the final action boundary even when model output is syntactically valid.

The agent is designed to be inspectable and finite, not universally autonomous. Task success, cost, latency, and failure modes require a separate public evaluation corpus before comparative claims are justified.

11. Model gateway

The OpenAI-compatible gateway resolves its API key on the trusted host. It applies a deadline to API-key-provider waiting and the HTTP exchange, and it enforces independent request-byte and response-byte ceilings.

Non-loopback model endpoints must use HTTPS. The gateway validates response structure and structured tool calls before they reach the agent. A model endpoint is not allowed to modify the browser session's origin, effect, approval, identity, or resource policy.

Compatibility with an OpenAI-style HTTP schema is an integration surface, not a claim that every provider implements identical semantics. Provider selection, data handling, retention, and billing remain deployment-owner decisions.

12. Qarinah context and provenance

The Qarinah recorder emits the versioned `cockroach.browser-memory.v2` envelope. It includes event type, session ID, optional actor, timestamp, a SHA-256 purposeDigest instead of the raw purpose, optional input and output digests, evidence identifiers, an optional receipt hash, and filtered descriptive metadata.

Recorder metadata is allowlisted. Eligible fields include action, status, input or output digest, receipt hash or identifier, evidence identifiers, policy digest, mode, effect, risk, and completion time. Values are bounded by depth, length, and count. Authorization, cookie, credential, password, passphrase, secret, token, storage, form-value, and API-key fields are removed recursively.

The envelope does not contain a source URL, raw page content, browser-profile material, hidden reasoning, or the raw session purpose. The host owns persistence. A recorder deadline defaults to 1,000 ms and can be configured within the implementation's bounds. Recorder rejection or timeout is reported operationally without replacing the browser result.

The context provider retrieves a bounded summary with citation identifiers and optional receipt and evidence anchors. The agent validates identifier sizes and citation counts, preserves anchors during truncation, and serializes the pack as an untrusted user-role observation behind a trusted system instruction.

Retrieved history can inform a proposal. It cannot create a session, add an origin, enable an action, approve an effect, supply a credential, relax a resource ceiling, or dispatch browser work. Memory never grants authority.

13. Maqam, crawler, and other integrations

Maqam supplies a separate governance proof. For a consequential operation, Cockroach Browser can produce canonical action material and a digest. Maqam can evaluate policy and bind approval to that exact material. Changed input, expired approval, or replay must fail. Browser evidence records what the runtime observed or executed; Maqam records why dispatch was authorized. Stable identifiers connect the two ledgers without pretending they are one transaction system.

Cockroach Crawler supplies breadth. It can map static HTTP content, feeds, documents, and public sources at bounded scale. Cockroach Browser supplies browser state, rendering, interactive operations, and evidence. A practical workflow crawls first, ranks candidate URLs, and opens only the pages that need a browser.

ProductLoop and provider integrations expose typed adapter contracts. Adapter status means another package, service, authority, or deployment is required. The presence of an adapter does not claim that Cockroach Browser operates the external system.

14. Evidence and receipts

Evidence can include semantic snapshots, screenshots, paired captures, Chromium PDFs, Playwright traces, HAR, video, console records, network metadata, downloads, audits, annotations, and action results. Evidence records are content-addressed and bound to the session that produced them.

An action receipt records canonical action material and digest, policy digest, URL before and after, result status, evidence identifiers, and the previous receipt hash. The linked chain makes missing, reordered, or modified terminal records detectable. Verification recomputes digests without replaying the browser.

Paired capture places a screenshot and a semantic snapshot under one reviewed page-state operation. Temporary annotations can label selected references without mutating application data.

Evidence proves what the configured runtime recorded. It does not prove that an external statement is true, that a human intended an action, that the website authorized automation, or that the runtime is free of vulnerabilities. Governance receipts and external facts require their own proof.

15. Team tenancy and admission

The daemon has an administrator identity and can be configured with unique actor bearer tokens. Actor-scoped tokens require TeamSessionStore; startup fails closed if actor tokens are configured without the ownership store. The administrator token and every actor token must be unique, or startup fails with AUTH_TOKEN_COLLISION.

An actor token cannot directly choose arbitrary session policy. Actor-created sessions additionally require a host-owned actorSessionFactory. That callback derives an authoritative session input from individually reviewed fields. The server replaces any caller-supplied actor with the authenticated identity and claims ownership before exposing the session. If the ownership claim fails, the newly created session is closed.

TeamSessionStore records one owner and revocable operator and viewer grants. Viewers inspect. Operators can use explicitly enabled action routes. Owners manage access and closure. Access mutations replace the in-memory map only after durable persistence succeeds, so a failed durable write leaves the prior access state in place. Raw profiles and cookies are not stored in access records.

Session admission is concurrency-safe. The daemon defaults to 32 global non-closed sessions and 8 non-closed sessions per actor. Pending creations reserve capacity so parallel requests cannot oversubscribe a limit. Limit responses use HTTP 429 and distinguish global from actor ceilings.

The activity stream is bounded and actor-filtered. Global health, Prometheus metrics, and evidence verification require the administrator identity. An OpenAPI index describes routes but does not grant permission to call them.

This is application tenancy for an owned deployment. It is not a claim of hostile multi-tenant browser isolation. Strong isolation between mutually untrusted tenants requires separate operating-system or container boundaries, secret domains, storage domains, and network policies.

16. Resource governance

For runtime-owned browser-server and lightweight launches whose process identifiers are observable, Cockroach Browser samples the complete owned process tree. It aggregates resident set size, cumulative CPU time, and process count and compares them with session ceilings. Duration is also enforced.

Sampling runs continuously and at action boundaries. On supported platforms, near-simultaneous sessions can reuse a fresh host process inventory to reduce telemetry overhead. The sampling interval is bounded. Windows uses a higher default because PowerShell and CIM process enumeration are comparatively expensive.

A detected resource breach, duration expiry, or telemetry loss becomes a sticky terminal error. It does not clear on a later lower sample. The runtime begins process-first teardown independently of the next browser action.

Successful close terminates the runtime-owned process before publishing terminal lifecycle state and releasing the remaining context, trace, and runtime records. If termination cannot be verified, the session returns TERMINATION_UNVERIFIED and remains retained for an explicit close retry. A persistent-profile writer lock also remains held rather than creating a second writer against an uncertain process.

Attached CDP and persistent headed contexts report process enforcement as unavailable when the runtime cannot establish ownership of the complete process tree. This is preferable to reporting a misleading zero or partial measurement.

Aggregate RSS is conservative and may count shared pages in more than one process. CPU time is cumulative process CPU time, not utilization percentage and not electrical energy. Polling can miss short-lived spikes or CPU from a child that exits between observations.

Application telemetry is not a real-time kernel boundary. Use Linux cgroups or container limits, Windows Job Objects, an operating-system sandbox, or equivalent infrastructure when a hard memory, CPU, process, or egress limit is required.

17. Network boundary

Governed full-engine sessions revalidate navigation, redirects, and intercepted HTTP or HTTPS subresources against session origin policy. Static fulfillment is bounded by rule and session byte ceilings. Runtime-owned full-engine contexts validate WebSocket handshakes and block service workers so that surface cannot silently bypass intercepted HTTP or HTTPS routing.

These controls do not contain WebRTC, STUN, TURN, UDP, WebTransport, QUIC, attached CDP traffic, lightweight-engine WebSockets, raw Playwright or Puppeteer traffic, or operator-owned protocol clients. Hostile content requires a deployment-owned operating-system, container, firewall, or equivalent deny-by-default egress boundary.

Webhook delivery uses public HTTPS destinations, revalidates DNS, rejects private or mixed-address results, preserves TLS hostname verification, and does not follow redirects. Signed webhooks use a separate delivery ledger and replay guard.

Headless mode is not a sandbox. Rendering, JavaScript, remote CDP, imported state, proxies, uploads, downloads, and browser extensions remain privileged capabilities.

18. Deployment architecture

The runtime can run in-process as a TypeScript library, as an authenticated local daemon, in a container, or behind an explicitly configured authenticated remote-worker boundary.

The default daemon binds to loopback and creates a strong bearer token. Non-loopback binding requires explicit remote mode, TLS, authentication, and a reviewed CORS allowlist. Raw HTTP action dispatch is disabled by default.

The included container path is intended for a Node runtime and real browser processes. A deployment should pin package and browser identities, run with a read-only root filesystem where practical, mount only required data and evidence paths, use a constrained temporary filesystem, and apply memory, CPU, PID, and egress limits derived from measured workloads.

BrowserWorkerPool can select reviewed authenticated workers by health, declared capacity, weight, and required tags. BrowserFleet can allocate local or external capacity with explicit engines and provider capabilities. These components do not turn the process-local job queue into a distributed transaction coordinator or create a managed global fleet.

The Cloudflare Worker or static-site deployment associated with project documentation is a documentation and distribution surface. It is not the Node-and-browser execution runtime and should not be marketed as browser execution at the edge.

The following local and container forms illustrate the intended boundary:

```
npx cockroach-browser serve --host 127.0.0.1 --port 43110
docker build -t cockroach-browser:0.5.0-rc.1 .
```

Remote deployment should add service identity, TLS termination, secret management, network policy, separate tenant isolation where needed, bounded artifact storage, monitoring, and a tested recovery path.

19. Distribution and release integrity

The primary software distribution is the npm package and CLI. Release candidates should also produce signed GitHub release artifacts with checksums, provenance, and a software bill of materials, plus pinned OCI images in a reviewed registry. Homebrew, Scoop, or winget distribution should follow only after repeatable signed binary releases exist.

MCP registry discovery and one-command setup for major coding-agent clients can reduce integration friction. Runnable TypeScript, Python, and Go examples should cover structured extraction, user-approved form submission, evidence-backed operations, and deliberate lightweight-to-full-engine routing.

The repository is AGPL-3.0-or-later. An enterprise distribution strategy should state clearly whether it remains pure copyleft or adds a separately reviewed commercial license. The paper does not set hosted-service pricing because an operated service, support boundary, tenancy model, and measured cost model are not established here.

The release-candidate identifier in this paper is not an immutable published tag. A production release should bind the reviewed commit, generated package, browser binaries, container image, paper, SBOM, checksums, and provenance before publication.

20. Verification status

Verification is layered. Schema and unit tests exercise policy, validation, receipts, structured extraction, resource telemetry, server tenancy, engine manifests, and agent behavior. Browser boundary tests exercise real engine processes where the host can launch them. Package tests consume the packed tarball rather than importing the source tree. Proof verification recomputes benchmark claims from retained artifacts.

A development-tree run associated with this release-candidate work reported 184 ordinary tests: 164 passed, 20 were skipped by their documented environment gates, and none failed. Separate Windows live managed-resource and Obscura lanes reported 18 of 18 passing. A Linux or WSL full-engine and raw-platform lane reported 28 passing with one intentionally skipped headed-only case. These results are a dated development snapshot, not an immutable release attestation. The final release candidate must rerun the complete gate on clean hosts and publish the logs or machine-readable reports.

The standard repository gate is:

```
npm ci
npm run check
```

The expanded gate includes type checking, build, tests, API-surface parity, lightweight-proof verification, package checks, site checks, production dependency audit, and tarball inspection. Real-browser and packed-consumer lanes should be executed explicitly in clean environments.

21. Lightweight benchmark question

The benchmark asks a narrow falsifiable question:

Can the complete runtime-owned Obscura browser process tree execute one pinned non-visual CDP, JavaScript, DOM, and form fixture, reject a visual action at preflight, and tear down correctly while every measured RSS observation remains at or below a stated target?

The target is evaluated jointly. A run passes only if every required conformance check succeeds and every retained owned-browser-tree RSS observation is at or below the target. A capability failure and a memory failure both fail the target.

The benchmark does not ask whether the whole Cockroach Browser application fits the target, whether arbitrary pages fit it, whether the engine renders accurately, whether full browsers fit it, or whether one product is faster or lighter than another.

22. Benchmark identity

The reviewed executable is obscura.exe, version Obscura 0.2.1, with a file size of 58,097,152 bytes.

The executable SHA-256 is:

```
5b609fb46bc00da79e450fb0fbd34bd442e565b1394f4af95433e0b341078221
```

The Git base commit is deb6c25b4c0bc1927a631e3b062464b4f4cc2775. The measured working tree was dirty, so the base commit alone is not the tested source identity.

The normalized source-tree SHA-256 is:

```
fb0c4597e39f319dd9b6f3bab02777c395e9d8d84906981bf939a39b470e7279
```

The runtime-build SHA-256 is:

```
6738efa4000ba482db83c9dc95ba2f21caed31de96f30dcd342e5dc722d86025
```

The benchmark-harness SHA-256 is:

```
08a5294f2d446765f712b93c9bfaaca010b1d043ded638d1f4f57b5038c97e86
```

Identity normalization uses forward slashes for relative paths and LF line endings for text. The source identity covers runtime, schemas, benchmark and package configuration, build configuration, and tests. Generated documentation and benchmark artifacts are excluded so that writing an output artifact does not change the measured input identity.

The recorded machine was Windows win32/x64 release 10.0.26200, with 16 logical CPUs, an AMD Ryzen 7 4800H, 16,557,887,488 bytes of total memory, and Node v24.15.0.

23. Benchmark method

Each target was run independently with one warmup and 20 measured launches. Each measured launch created one runtime-owned loopback CDP server and loaded one non-visual data-URL document.

The required operations were connection, JavaScript evaluation, DOM query, text input, and HTML element click dispatch. The required checks were CDP connection, JavaScript, DOM, forms, screenshot preflight denial, and teardown. Every required check passed in every measured launch for both targets.

The measurement aggregated RSS across the complete runtime-owned browser process tree from spawn through startup, CDP connection, workload, settle, and shutdown preparation. The Node coordinator was sampled and reported separately.

Each measured launch included 10 explicit steady-state samples at 25 ms intervals plus boundary and continuous observations. The 30 MiB target retained 478 timestamped process-tree observations across its 20 measured launches, and the 25 MiB target retained 480, for 958 observations across both targets.

The timing bounds were 15,000 ms for startup, 10,000 ms for an action, 5,000 ms for teardown, and 500 ms for settle.

Energy was unavailable because the host did not provide a calibrated energy counter. CPU time is not reported as watts.

The renderingPolicy value visual-actions-denied means that Cockroach Browser rejected screenshot capture during preflight. It does not assert that the selected engine disabled or omitted rendering.

24. Benchmark results

The 30 MiB target was 31,457,280 bytes. It passed. The per-launch peak distribution was:

- Minimum: 28,893,184 bytes.
- Median: 29,347,840 bytes.
- p95: 29,569,024 bytes.
- Maximum: 29,622,272 bytes, or 28.25 MiB.
- Required capabilities: all passed.

The 25 MiB target was 26,214,400 bytes. It failed because the owned browser process tree exceeded the target, while every required capability check still passed. The per-launch peak distribution was:

- Minimum: 28,831,744 bytes.
- Median: 29,323,264 bytes.
- p95: 29,634,560 bytes.
- Maximum: 29,679,616 bytes, or approximately 28.30 MiB.
- Required capabilities: all passed.

The passing artifact is

docs/benchmarks/artifacts/obscura-0.2.1-constrained-non-visual-30mib-2026-09-03-rc1.json.

Its SHA-256 is:

```
f90b31d6f5d5096300ac2722ed835db0483a76dc4d51ee85e86604a6634c0aa7
```

The failing artifact is docs/benchmarks/artifacts/obscura-0.2.1-constrained-non-visual-25mib-2026-09-03-rc1.json.

Its SHA-256 is:

```
581eb93577d6b52c71e02d7e0b71914f88acd0920a6e0e06925aae0a4575d2df
```

The [canonical proof record](#) and the two immutable JSON artifacts are the source of truth for individual observations, coordinator measurements, timings, conformance, and teardown records. The September 2 five-run record is historical and superseded.

25. Interpreting the result

The supported external statement is:

"The pinned Obscura 0.2.1 constrained non-visual fixture reached a maximum complete owned-browser-process-tree RSS of 29,622,272 bytes, or 28.25 MiB, across 20 measured launches and passed the 30 MiB target. A separately measured 25 MiB target failed with a 29,679,616-byte maximum."

That statement should travel with the binary digest, host, workload, process-tree definition, sample count, conformance result, failing target, and coordinator exclusion.

The result demonstrates that a compatible DOM-oriented lane can operate below a 30 MiB owned-browser-tree target for this fixture. It does not demonstrate a 28 MB product footprint. It does not make a promise about an arbitrary workload. Full engines normally use much more memory under realistic rendered workloads.

The honest product formulation is:

"Full browsers when fidelity matters. A measured 28.25 MiB non-visual lane when it does not."

26. Competitive landscape

The following landscape was checked against official public surfaces on 2026-09-03. Products change, so these descriptions should be revalidated before each external publication.

[Playwright](#) supports Chromium, Firefox, and WebKit and supplies broad automation, testing, tracing, code generation, and isolation capabilities. [Playwright MCP](#) provides an official MCP surface. Multi-engine control and MCP are therefore ecosystem parity, not exclusive Cockroach Browser claims.

[Puppeteer](#) provides Chrome and Firefox automation using CDP and WebDriver BiDi surfaces. Cockroach Browser's Puppeteer, CDP, tracing, screenshot, PDF, and protocol access should be presented as interoperability and operator reach.

[Selenium WebDriver](#) is a long-established cross-browser standards-based automation interface. Cockroach Browser's WebDriver and Appium routes extend operator connectivity; they do not replace the Selenium ecosystem.

[Browser Use](#), [Stagehand](#), and other agent-browser frameworks expose model-directed browser workflows. A built-in agent loop is table stakes. Cockroach Browser's position is the authority, capability, resource, and evidence boundary around the loop.

[Browserbase](#), [Browserless](#), [Steel](#), and [Cloudflare Browser Run](#) offer different combinations of managed browser capacity, automation protocols, observability, profiles, proxies, extraction, and agent integration. Cockroach Browser does not claim comparable managed global scale. Its near-term position is customer-owned execution, multi-engine routing, explicit authority, and reproducible evidence.

[Obscura](#) is the upstream independent lightweight engine used in the measured lane. Its [0.2.1 release](#) documents evolving rendering and automation behavior. Cockroach Browser claims the pinned adapter, policy, preflight, process ownership, conformance fixture, and independent measurement, not invention of Obscura.

[Lightpanda](#) is an independent lightweight browser project with CDP-oriented automation. Its [published benchmark documentation](#) covers broader workloads than the single Cockroach Browser fixture. Cockroach Browser does not compare these results directly because versions, hosts, workloads, concurrency, and memory definitions differ.

Within this reviewed set, per-action engine manifests, memory that is explicitly non-authoritative, continuous process-tree governance, and build-linked proof with a published failing target form an uncommon combination. "Uncommon" is a dated positioning observation, not a universal exclusivity claim.

27. Intended users and jobs

The primary audience is engineers building AI-agent platforms or operational automation that must remain self-hosted, respect data-residency constraints, limit model authority, or retain inspectable evidence.

Data extraction and crawling teams are a secondary audience when DOM-oriented jobs are constrained by memory or concurrency and can be routed away from a full rendering engine.

QA and developer-tool teams are a secondary audience when they need full Chromium, Firefox, and WebKit fidelity plus an agent-facing governed layer.

Regulated operations teams are a secondary audience when a workflow needs explicit approvals, bounded effects, ownership, and evidence retention. The software does not itself confer regulatory compliance.

The initial wedge is not consumer desktop browsing, a turnkey global proxy or CAPTCHA service, or the cheapest hosted Chrome hour.

The principal jobs are:

- Let an agent inspect and operate a website without inheriting ambient profiles, credentials, origins, or machine authority.
- Select a lightweight DOM lane or a full-fidelity browser lane before launch.
- Reject unsupported work instead of silently downgrading or swapping engines.
- Reuse Playwright, Puppeteer, CDP, BiDi, WebDriver, Appium, MCP, and existing browser knowledge.
- Enforce finite RSS, CPU-time, duration, action, evidence, and byte budgets.
- Produce evidence and receipts that an operator can inspect.
- Run locally or in customer-controlled infrastructure with authenticated team access.
- Supply bounded cited context without turning retrieved history into browser authority.

28. Adoption and go-to-market

The first product message should lead with the category and two-lane architecture, followed immediately by a runnable quickstart and the exact proof:

"One governed browser runtime for AI agents. Route compatible DOM work through a measured 28.25 MiB Obscura lane, use Chromium, Firefox, or WebKit for full fidelity, and enforce origins, effects, approvals, resources, and evidence before the model acts."

The next visible features should be engine preflight, fail-closed experimental opt-in, session authority, verified teardown, Playwright and Puppeteer interoperability, MCP and SDK access, evidence and receipts, Qarinah provenance, and customer-owned deployment.

Distribution should use:

- npm and the CLI for the primary installation path.

- Signed GitHub releases with checksums, provenance, and an SBOM.
- Pinned GHCR or Docker Hub images after container verification.
- An MCP registry listing and client-specific setup guides.
- Runnable quickstarts in TypeScript, Python, and Go.
- Templates for extraction, reviewed form submission, evidence-backed operations, and lightweight-to-full routing.
- GitHub releases, Discussions, issues, and a public compatibility board.
- A benchmark page that exposes raw artifacts and exact reproduction commands.
- A concise architecture article and a separate benchmark article derived from this paper.
- Show HN, Product Hunt, browser-automation communities, scraping communities, local-AI communities, agent-framework communities, and direct design-partner outreach after clean-host installation works.

The first 30 days should reconcile version, capability count, benchmark scope, limitations, and release commands across package metadata, README, paper, site, and release assets. Clean installation, first session, proof verification, and uninstall should be repeatable.

Days 31 through 60 should add same-host pinned comparisons, a real-page compatibility corpus, concurrency testing, a long soak, fault injection, security-negative tests, and integrations with at least three agent frameworks.

Days 61 through 90 should publish the paper, raw results, known limitations, signed artifacts, and runnable demos together; recruit design partners with explicit permission for any public reference; and maintain an evidence and compatibility changelog.

Useful operating metrics are clean-install success rate, median time to first verified session, benchmark reproduction rate, weekly active projects, four-week retained projects, task success by engine, orphan-process rate, resource-limit enforcement rate, and issue time to resolution. These are proposed metrics, not current adoption claims.

29. Limitations and open risks

The current lightweight proof covers one deterministic non-visual fixture on one Windows machine. It does not cover real-page compatibility, visual correctness, concurrency, long-running memory growth, energy, broad latency, or agent task success.

Obscura is experimental. Its CDP and web-platform behavior is not assumed to match Chromium. Rendering, screenshots, PDF, frames, Shadow DOM, storage, downloads, and other declared experimental capabilities need version-pinned conformance before release claims.

Lightpanda managed launch is intentionally unavailable until a complete engine-level or operating-system-level egress boundary covers relevant protocols and workers.

The application network boundary does not contain WebRTC, STUN, TURN, UDP, WebTransport, QUIC, attached CDP, lightweight-engine WebSockets, or raw operator traffic.

Process sampling can miss brief spikes and is not equivalent to cgroups, Job Objects, or another kernel boundary. Attached CDP and persistent headed contexts can make complete process ownership unavailable.

The built-in queue is process-local and file-backed. It is not a distributed consensus or transaction system. BrowserWorkerPool routes capacity but does not create global scheduling correctness.

The agent loop is finite and validated, but model output can still be wrong. Page content and retrieved context can attempt prompt injection. Runtime policy, exact approval, output bounds, citations, and fresh snapshots reduce authority and ambiguity; they do not make model reasoning correct.

Evidence can be altered outside protected storage unless deployment controls preserve it. Hashes detect changes relative to a trusted head and artifact identity; they do not prevent deletion or compromise of every surrounding system.

Browser automation may violate a site's rules or a user's authority even when the software can execute it. Operators must obtain authorization and respect applicable terms, privacy requirements, rate limits, and law.

29.1 Windows SideBySide limitation

On the Windows verification host used during release-candidate testing, the installed Playwright Firefox executable and the installed full Chrome executable failed before Cockroach Browser could establish browser control. Windows reported a SideBySide private-assembly resolution problem. The Firefox event identified a missing mozglue assembly version 1.0.0.0. The Chrome event identified a missing versioned assembly 151.0.7922.34.

This is a host-specific packaging or assembly-resolution limitation, not evidence that Firefox or Chrome generally fails and not evidence of a Cockroach Browser policy failure. Playwright Chromium headless shell and the pinned Obscura path passed on the Windows host, while the full-engine and raw-platform suite passed on Linux or WSL.

Windows private assemblies and the assembly search process are described in Microsoft's [About Private Assemblies](#) and [Assembly Searching Sequence](#) documentation.

The release should either resolve the local browser installation and rerun the affected Windows lanes, or publish the exact host limitation and supported Windows route. A passing Linux or WSL lane must not be relabeled as native Windows coverage.

30. Reproducibility

Verify the checked-in proof record, artifact hashes, narrative bindings, and current source and build identities with:

```
npm run verify:lightweight-proof
```

To produce new evidence, build the same reviewed source identity, obtain the exact Obscura binary whose SHA-256 is recorded above, and run each target independently. Replace the executable path with the verified local path.

```
npm run build
node scripts/benchmark-lightweight.mjs --executable C:/reviewed/obscura.exe --implementation
obscura --sha256 5b609fb46bc00da79e450fb0fbd34bd442e565b1394f4af95433e0b341078221 --resource-
profile constrained --target-mib 30 --warmup 1 --iterations 20 --resource-samples 10 --resource-
sample-interval-ms 25 --settle-ms 500 --startup-timeout-ms 15000 --action-timeout-ms 10000 --
teardown-timeout-ms 5000 --output output/resource-benchmarks/obscura-30mib-new.json
node scripts/benchmark-lightweight.mjs --executable C:/reviewed/obscura.exe --implementation
obscura --sha256 5b609fb46bc00da79e450fb0fbd34bd442e565b1394f4af95433e0b341078221 --resource-
profile constrained --target-mib 25 --warmup 1 --iterations 20 --resource-samples 10 --resource-
sample-interval-ms 25 --settle-ms 500 --startup-timeout-ms 15000 --action-timeout-ms 10000 --
teardown-timeout-ms 5000 --output output/resource-benchmarks/obscura-25mib-new.json
```

The failing 25 MiB command should write its JSON report and return a non-zero exit. Preserve that non-zero verdict. A new run is new evidence and must not overwrite the immutable September 3 artifacts.

For release conformance, use clean pinned environments and retain the output from:

```
npm ci
npm run typecheck
npm run build
npm test
npm run api-surface:check
npm run verify:lightweight-proof
npm run check:package
npm run check:site
npm audit --omit=dev
npm pack --dry-run --ignore-scripts
```

Run real-browser tests on supported clean Windows, Linux, and macOS hosts. Record browser versions, operating-system build, CPU architecture, machine memory, Node version, command line, skips, failures, and artifact hashes. A release attestation should identify exact logs rather than reporting only a total.

31. Evidence roadmap

The current result is a rigorous scoped proof, not an overall performance or production-readiness verdict. Evidence should expand in this order.

- Release identity: signed source, package, container, browser, harness, paper, SBOM, and provenance bindings.
- Correctness: per-engine bounded-action tests, parser properties, protocol conformance, and deterministic fixtures.
- Comparable performance: same-host browser-tree RSS or PSS, CPU, latency, throughput, and separately reported coordinator overhead against pinned baselines.
- Compatibility: at least 100 documented public pages plus deterministic SPA, frame, form, download, authentication, and challenge fixtures.
- Reliability: a 24-hour soak, thousands of launches, crash and hang classification, memory-growth slope, and zero unexplained owned-process orphans.
- Concurrency: 1, 10, 25, 50, and 100 sessions with admission behavior and saturation disclosed.
- Agent quality: public task success, cost, latency, step count, and failure taxonomy.
- Security negatives: SSRF, redirects, DNS rebinding, WebSockets, service workers, private networks, prompt injection, secret redaction, token scope, path traversal, persistence failure, and teardown failure.
- Platforms: Windows, Linux, macOS, Docker, and supported architectures with failures published beside passes.
- Independent review: third-party benchmark reproduction, threat-model review, and security assessment.

Claims should expand only after the corresponding evidence exists.

32. Conclusion

Cockroach Browser is designed to make browser capability usable by AI agents without silently transferring ambient machine authority. It combines full browsers for fidelity, an explicitly experimental lightweight lane for compatible work, preflighted engine capabilities, bounded sessions, finite resources, structured observations, model integration, cited context, tenancy, and inspectable evidence.

The most important measured result is also deliberately narrow: a pinned Obscura 0.2.1 non-visual fixture passed a 30 MiB complete owned-browser-process-tree target at a 29,622,272-byte maximum, while a separate 25 MiB target failed at 29,679,616 bytes. Publishing both outcomes defines the present boundary more accurately than a rounded product promise.

The path to adoption is therefore evidence-led. Make setup fast, make engine selection explicit, make unsupported work fail before launch, make authority inspectable, make teardown uncertainty visible, and publish artifacts that others can verify.

Appendix A. Bounded action kinds

The 66 bounded action kinds in 0.5.0-rc.1 are:

- navigate
- back
- forward
- reload
- click
- doubleClick
- fill
- type
- press
- hover
- focus
- check
- uncheck
- select
- scroll
- drag
- mouse.move
- mouse.down
- mouse.up
- mouse.click
- keyboard.down
- keyboard.up
- keyboard.insertText
- upload
- download
- evaluate
- query.inspect
- emulation.set
- emulation.clear
- cache.clear
- console.clear
- network.clear
- wait
- challenge.resolve
- history.inspect
- capture.paired
- annotate.show

- `annotate.clear`
- `clipboard.read`
- `clipboard.write`
- `network.inspect`
- `network.export`
- `network.route.add`
- `network.route.remove`
- `network.routes.list`
- `state.save`
- `state.load`
- `state.list`
- `state.delete`
- `screenshot`
- `pdf`
- `snapshot`
- `extract`
- `extract.structured`
- `cookies.read`
- `cookies.write`
- `storage.read`
- `storage.write`
- `tab.open`
- `tab.close`
- `tab.switch`
- `tab.lock`
- `tab.unlock`
- `tab.lock.status`
- `trace.start`
- `trace.stop`

Appendix B. Claim controls

Approved category wording:

"A governed, local-first, multi-engine browser runtime for AI agents."

Approved measured hero wording:

"Full browsers when fidelity matters. A measured 28.25 MiB non-visual lane when it does not."

Approved routing wording:

"Use the lightweight lane for compatible DOM and JavaScript work, and Chromium, Firefox, or WebKit for full fidelity."

Prohibited or unsupported wording includes "28 MB or less," "every browser feature in 28 MB," "the only," "the first," "no other browser," "faster than Chrome," "lighter than Lightpanda," "secure sandbox," "complete isolation," "all egress is contained," "works on every site," "unblockable," "bypasses CAPTCHAs," and "renderer disabled."

Claims about operated regions, proxies, CAPTCHA services, stealth infrastructure, customer counts, compliance, reliability, savings, or a hosted fleet require current operational evidence and must not be inferred from adapter interfaces.

Appendix C. Release and deployment checklist

- Bind the immutable release tag to the reviewed commit.
- Regenerate the source-derived capability inventory and confirm its counts.
- Run clean Node 22, 24, and 26 gates.
- Run full-engine browser tests on supported clean hosts.
- Resolve or disclose the Windows SideBySide limitation.
- Verify the canonical benchmark and retain both pass and fail artifacts.
- Inspect package contents from the packed tarball.
- Pin browser versions and record their digests where available.
- Generate checksums, provenance, signatures, and an SBOM.
- Build and scan the OCI image.
- Exercise loopback daemon authentication and remote TLS rejection paths.
- Exercise actor ownership, revocation, persistence failure, and admission ceilings.
- Exercise resource breach, telemetry loss, and unverified teardown paths.
- Exercise network-negative cases and document protocols requiring OS containment.
- Verify every external link and every local paper link.
- Render and visually inspect the paper without modifying the v1.1 artifacts.
- Publish limitations beside the quickstart and benchmark result.

Appendix D. Source and evidence map

- Source repository: github.com/AjnasNB/cockroach-browser
- Public documentation: cockroachbrowser.com
- npm package: npmjs.com/package/cockroach-browser
- Paper series DOI: [10.5281/zenodo.21701791](https://doi.org/10.5281/zenodo.21701791)
- Canonical lightweight proof: docs/benchmarks/obscura-non-visual-2026-09-03.md
- Headless compatibility guide: docs/headless-compatibility.md
- Resource-governance guide: docs/resource-governance.md
- Market-positioning research: docs/market-positioning.md
- Qarinah integration guide: docs/qarinah.md
- Deployment guide: docs/deployment.md
- Security guide: docs/security.md

Acknowledgements

The author thanks Shahin Ahammed for contributions to product direction, use-case definition, positioning, and review of the manuscript. Cockroach Browser builds on the work of browser-engine, automation, protocol, parsing, and open-source communities cited in this paper.

References

- 1 Cockroach Browser source and documentation, github.com/AjnasNB/cockroach-browser.
- 2 Cockroach Browser technical-paper series, [concept DOI 10.5281/zenodo.21701791](https://doi.org/10.5281/zenodo.21701791).
- 3 Cockroach Browser canonical Obscura proof, [September 3, 2026 record](#).
- 4 Microsoft, [Playwright browser documentation](#).
- 5 Chrome Browser Automation team, [Puppeteer introduction](#).
- 6 Selenium project, [WebDriver documentation](#).
- 7 Microsoft, [Playwright MCP](#).
- 8 Model Context Protocol, [specification and documentation](#).
- 9 Chrome DevTools Protocol, [protocol documentation](#).
- 10 Obscura, [source repository](#) and [0.2.1 release](#).
- 11 Lightpanda, [documentation](#) and [benchmark methodology](#).
- 12 Browser Use, [cloud quickstart](#).
- 13 Stagehand, [introduction](#).
- 14 Browserless, [product documentation](#).
- 15 Steel, [product documentation](#).
- 16 Cloudflare, [Browser Run documentation](#).
- 17 Microsoft, [About Private Assemblies](#).
- 18 Microsoft, [Assembly Searching Sequence](#).
- 19 WHATWG, [HTML Living Standard](#).
- 20 WHATWG, [URL Living Standard](#).
- 21 IETF, [RFC 9110: HTTP Semantics](#).
- 22 IETF, [RFC 6454: The Web Origin Concept](#).
- 23 Maqam, [source and documentation](#).
- 24 Qarinah, [technical documentation](#).
- 25 Cockroach Crawler, [source and documentation](#).